

Real-Time Topology-Aware Log–Metric Correlation for LLM-based Root Cause Analysis in Distributed Systems

Preetham M, Rahul Singh, Jit Gupta, Tarun Banka

Juniper Networks

{mpreetham, rahulsingh, gjit, tbanka}@juniper.net

Abstract— Modern enterprise infrastructures increasingly rely on microservices, hybrid multicloud platforms, and distributed networks, making cross-layer root cause analysis (RCA) a significant challenge. System failures often span the application, platform, and network layers, yet conventional metric-based RCA approaches are limited in their ability to associate anomalous metrics with the high volume of unstructured and heterogeneous log data. Although logs contain valuable diagnostic context, their scale, diversity, and temporal misalignment hinder effective correlation, leading to prolonged mean time to resolution (MTTR). This paper presents a real-time, topology-aware framework for log–metric correlation in hybrid cloud and distributed network environments. The proposed framework employs multi-dimensional scoring to annotate log events and integrates a correlation engine that aligns these events with anomalous performance metrics using topology constraints and temporal synchronization. A Large Language Model–driven module, referred to as RCA Copilot, leverages these correlated signals to generate succinct, human-readable diagnostic summaries and to interpret cross-layer anomalies. By facilitating critical log retrieval, cross-layer dependency awareness, and structured, template-based statistical analysis, the framework provides actionable insights that accelerate fault diagnosis and resolution. Through efficient temporal alignment and integration of heterogeneous log sources, it enhances diagnostic precision, reduces MTTR, and supports autonomous, explainable observability across hybrid multicloud environments.

Keywords— Log Analytics, Observability, Hybrid multicloud, Log–Metric correlation, AI workloads, LLM, RCA

I. INTRODUCTION

Modern enterprise infrastructures span microservices, hybrid multicloud platforms, and distributed network layers, where faults propagate across components and complicate cross-layer root cause analysis (RCA). While performance metrics offer fast anomaly detection, they lack semantic depth and cannot capture the fine-grained behaviours preceding failures. Logs contain these diagnostic cues but arrive at massive scale with heterogeneous formats and no topology context making real-time log-metric correlation difficult and error-prone. Existing observability pipelines therefore struggle to align unstructured logs with metric anomalies or to trace failures across distributed layers, leading to incomplete diagnostics and elevated MTTR. LLM-based RCA systems require compact, structured, correlation-ready signals, as direct ingestion of raw logs is infeasible due to scale, latency, and token constraints, exposing a gap between metric-only RCA and unstructured log analytics. These challenges motivate the need for a unified framework

capable of (i) structuring heterogeneous logs at scale, (ii) correlating log patterns with anomalous performance metric behaviours in real time, and (iii) exposing compact, high-value diagnostic context consumable by LLM agents. Such capabilities are essential for accurate fault localization and reliable operations in modern hybrid cloud environments.

II. SOLUTION OVERVIEW

To address these limitations, we propose a real-time, topology-aware log analytics framework designed for integration with LLM-based RCA systems. The framework treats logs and metrics as complementary diagnostic sources and performs correlation to identify causal relationships between log events and observed anomalies. Its features are as follows:

1. Log summarization that condenses millions of unstructured raw logs into structured templates and log-rate timeseries, enabling cross-layer analytics.
2. Multi-dimensional scoring to detect critical log entries for end-to-end observability of network.
3. Real-time metric-to-log alignment for detecting anomaly-related log templates across topology layers.
4. An LLM-driven RCA Copilot that interprets correlations and generates explanations with actionable insights.

Our framework also supports distributed AI/ML workloads on GPU clusters by analysing NVIDIA's Collective Communication Library (NCCL) [1] communication logs and correlating them with system-level anomalies for precise fault localization. Additionally, it provides anomaly-focused log retrieval, critical-pattern detection, and structured log-template time-series generation. This system significantly reduces MTTR and improves the reliability and explainability of RCA in large-scale infrastructures on hybrid multicloud Kubernetes deployments. To show the generalized nature of our framework, all theoretical part is shown in hybrid cloud environment while the results section uses an on-prem AI workload environment.

III. ARCHITECTURE

The proposed framework implements a three-stage pipeline in Fig. 1, that transforms raw, heterogeneous log data into structured, context-rich diagnostic insights. This pipeline is optimized for scalability, latency, and compatibility with LLM-based RCA systems.

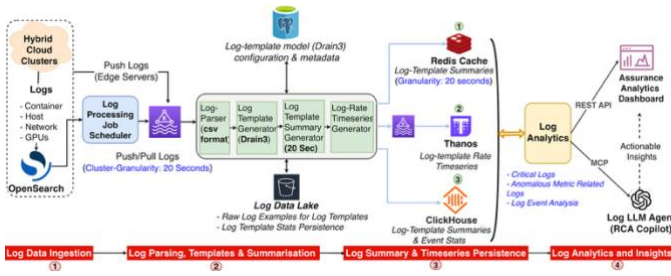


Fig. 1. Log Analytics Pipeline

Stage 1 – Log Ingestion and Summarization Pipeline:

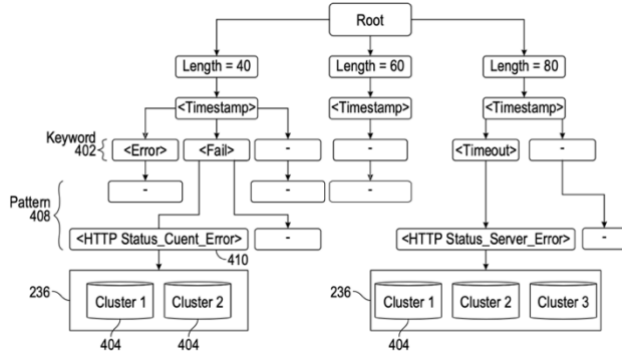


Fig. 2. Log Template Drain Parser Tree

The first stage of the framework performs log ingestion and reduction of heterogeneous logs from application services, Kubernetes hosts, and network devices. Logs are collected in micro-batches (≈ 10 -20-seconds granularity) using topology-aware routing where each log is tagged with source topology metadata (ex. layer, node ID, service name) and directed to layer-specific parsers. An ETL pipeline parses, normalizes and structures them into template-based summaries using an enhanced Drain3 [2] engine, converting templates into log-rate time series stored in a NoSQL datastore for low-latency access. To improve template precision and downstream correlation accuracy, Drain3 is augmented with a keyword- and pattern-aware routing layer (as shown in Fig. 2). Incoming logs are evaluated for critical lexical cues (e.g., *error*, *fail*, *OOM*) and structured patterns (e.g., HTTP codes, network failure signatures). These features drive the construction of a keyword and pattern-aware prefix tree, where branches encode semantically meaningful combinations rather than generic tokens. Logs are routed to the best-matching leaf based on structural and semantic features, yielding fine-grained templates that avoid over-generalization, improve log-rate signal quality, and strengthen cross-layer log-metric correlation.

Stage 2 – Cross-Layer Log Analytics Pipeline: The second stage performs correlation between preprocessed log template summaries, log-rate timeseries and performance metrics obtained from timeseries databases via Dynamic Time Warping (DTW) [3] time-series alignment techniques. Metric anomalies are correlated only with logs from topology-constrained dependency paths, such as analysing backend services, hosts, and network devices when a frontend latency anomaly is detected. It generates structured diagnostic signals that describe

"what failed," "where," and "when". It produces summarized analytics outputs accessible via REST APIs or an MCP (Model Context Protocol) [4] server. These outputs serve as high-fidelity inputs for the LLM based RCA system, avoiding the cost and noise of raw log analysis.

Stage 3 – LLM-Powered RCA Log Agent: The final stage introduces a large language model-driven log agent that consumes the structured diagnostics from Stage 2. It performs context-aware reasoning to infer probable root causes, generate diagnostic summaries, and recommend corrective actions. By leveraging LLMs on curated, context-rich data rather than unprocessed logs, this stage delivers explainable, actionable RCA insights while maintaining computational efficiency. These stages enable scalable log processing, real-time metric correlation, and LLM-assisted diagnosis, significantly reducing Mean-Time-To-Resolution (MTTR) and improving operational visibility across distributed infrastructures.

IV. LOG-METRIC CORRELATION ANALYSIS

In multi-layered environments temporal misalignment between metrics and logs makes cross-layer correlation difficult. Metrics arrive with sub-second latency while logs incur seconds-to-minutes ingestion delays, and asymmetric temporal patterns (e.g., short metric spikes versus prolonged log bursts or delayed critical events) render strict time-aligned methods such as Pearson correlation [5] ineffective. These limitations prevent accurate causal analysis, underscoring the need for delay-tolerant, cross-layer log-metric correlation methods.

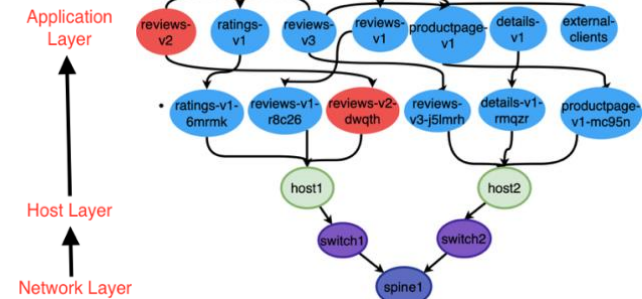


Fig. 3. Knowledge Graph for Three-layer Topology

In Fig. 4 we explain the cross-layer log-metric correlation, where we consider a three-layer topology with application, hosts, and network layers, similar to Fig. 3. During an RCA time window, metric anomalies propagate across these layers. For correlation, the system selects logs from the subgraph centred on the anomalous application node identified through abnormal KPI behaviour. By restricting analysis to logs from directly dependent nodes instead of the entire knowledge graph, the framework significantly reduces the number of candidate logs without reducing RCA accuracy. The correlation analysis proceeds through the following high-level steps:

Step 1- List of performance metric anomalies as an input: Cross-layer performance metric data is independently analyzed for anomalies using machine-learning-based methods on the underlying dependency graph. The log analysis phase then begins by collecting the performance metric anomalies detected

along with the corresponding topology information like layers and timestamps when these anomalies were observed.

Step 2- Reduce log dataset for analysis using topology-based filtering: Based on the metric anomalies detected in Step 1, a subgraph is derived from the knowledge graph using the anomalous application node as the anchor. Only logs from nodes within this subgraph and their directly connected neighbors are considered for analysis. This topology-aware filtering significantly reduces the search space without compromising accuracy. Logs for the selected nodes are retrieved from the log-template summary stores which are implemented using a combination of cache and NoSQL storage systems maintain 20-second-granularity template summaries along with the latest raw log examples. The analysis window accounts for ingestion delays and temporal offsets between metric anomalies and log generation across layers, ensuring a focused and time-aligned dataset for correlation.

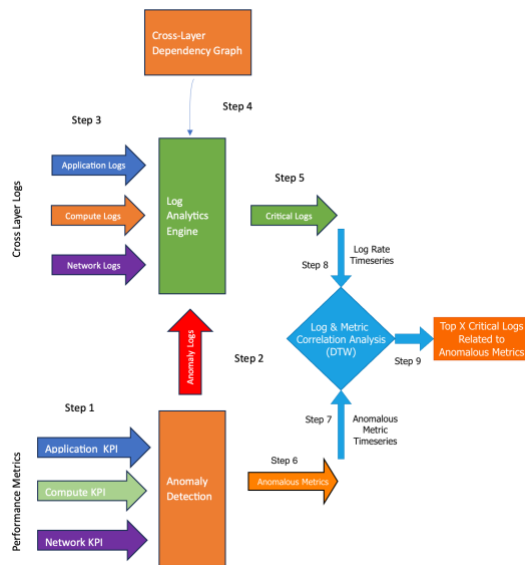


Fig. 4. Log-Metric Correlation Process

Step 3- Multi-Dimensional Scoring for Top K Critical Log Templates: Critical event identification requires automated prioritization that captures diagnostic signals while filtering operational noise. Our log scoring framework integrates five signals: (1) Temporal recency with exponential decay weighting prioritizes recent logs, (2) Critical keyword presence (ERROR, FATAL, OOM, TIMEOUT, EXCEPTION) boosts error-indicative entries, (3) Log rarity scoring using inverse document frequency (IDF) highlights anomalous patterns, (4) Unrecognized template detection flags novel log structures indicating zero-day issues, and (5) Noise filtering penalizes verbose informational logs (INFO, DEBUG) to prevent diagnostic dilution. Each log template is assigned a `critical_score` (as shown in Eq 1), derived as the combined weight of recency, rarity, category significance, and insignificance penalties. Scores are normalized using a softmax function to obtain a probability distribution over all templates. The top K templates with the highest probabilities are selected as the most critical diagnostic logs.

$$\text{Critical Score} = \text{Time Recency} * \text{Log Rarity} * \text{Keyword score} * \text{Log Insignificance score} \quad \text{Eq 1.}$$

Step 4- Log-Metric Correlation Using Constrained DTW: After topology-based filtering and scoring, relevant logs are mapped to their corresponding log-template timeseries. Since each node may emit multiple templates, n template-specific log-rate timeseries are retrieved from the timeseries datastore (e.g., Thanos) for the analysis window. These series capture the temporal frequency of each log pattern. To determine which logs best explain the metric anomaly, Dynamic Time Warping (DTW) [3] is used to compute similarity between the anomalous metric timeseries and each of the n log-template series. DTW is well-suited for this task because it provides non-linear temporal alignment, enabling flexible many-to-many matching that naturally handles temporal misalignment, ingestion delays, and phase shifts across layers. This robustness allows DTW to detect correlations even when metric spikes and log bursts differ in duration or appear in inverted causal order. For all candidate templates, DTW distances are computed and ranked; the Top-N templates with the smallest distances are selected as the most strongly correlated log patterns. To ensure scalability, we employ constrained DTW with Sakoe-Chiba band [6], reducing time complexity from $O(|M| \cdot |L|)$ to near linear $O(|M| \cdot w)$ where w bounds temporal warping. We set $w = 10$, allowing ± 5 interval shifts, which captures realistic ingestion delays. Additionally, Top-K early termination halts DTW evaluation for candidates whose partial distance already exceeds the current k -th best score. These templates provide actionable, cross-layer insights into the anomaly's underlying root cause.

Step 5- Returning Top K Correlated Logs via HTTP REST API and MCP Server: The analytics pipeline delivers near real-time results via an HTTP POST API and an MCP (Model Context Protocol) interface. Given the network topology, anomalous nodes, and analysis window, it returns the Top-K cross-layer correlated log templates with representative log examples and associated layer and topology metadata for each detected anomaly.

V. APPLICATION IN LLM-POWERED RCA FOR ACTIONABLE INSIGHTS

Direct integration of raw logs with an LLM is infeasible: a 5-minute window can produce millions of log lines ($\approx 15M$ tokens), far exceeding context limits and making inference prohibitively expensive. Our framework acts as an intelligent pre-processing layer, reducing context via template clustering ($\sim 7,500$ tokens) and DTW-based correlation filtering ($\sim 1,000$ tokens for Top-K templates), enabling LLM-driven diagnostics without losing critical signals. The LLM-powered Diagnostic Agent consumes these filtered, correlated logs enriched with topology metadata, anomaly metrics, and statistical RCA outputs to construct unified prompts for the RCA Copilot. The Copilot generates structured diagnostics, including human-readable failure explanations, critical log-pattern identification, evidence-based root-cause reasoning, and actionable remediation guidance. A semantic validation step ensures DTW-correlated logs meaningfully explain observed metric anomalies. By restricting LLM inputs to only the most critical

signals per window, the system preserves diagnostic fidelity, reduces resolution latency, and avoids LLM context-window limitations.

VI. EVALUATION AND COMPARATIVE ANALYSIS

Template-based summarization achieves over an order-of-magnitude reduction in log storage and enables sub-second RCA queries. By storing a single representative raw log per Drain3 template every 20–30 s summary per node or layer, the framework maintains compact metadata, accelerates log–metric correlation, and provides LLM-based RCA agents with concise, actionable context, reducing MTTR. Existing observability and AIOps systems (commercial or open-source) offer basic log-metric correlation using timestamps, service IDs, or labels but remain largely metric-driven, rely on keyword or trace-ID searches, and provide limited cross-layer causal reasoning. In contrast, our framework delivers topology-aware, temporally elastic log-metric correlation and exposes structured, LLM-ready diagnostic context (Table I).

TABLE I. RAW LOG STORAGE VS. TEMPLATE-BASED SUMMARIES

Aspect	Raw Logs (Search-Based)	Template Summaries (Proposed)
Stored content	Full Raw / indexed logs	Template Summaries + Log-Rate Series + Sample Log
Storage Reduction (100 GB raw)	1x (uncompressed, no reduction)	100-200x reduction (~0.5-1 GB)
Per-node log growth	High and unbounded	Bounded and topology-aware
Query Latency	Seconds-minutes	Milliseconds-seconds
Cross-Layer Correlation	Limited, timestamp/label-based	Topology-aware, temporally elastic
Suitability for LLM-based RCA	Poor (token-heavy)	High (compact, structured, context-rich)
Diagnostic Fidelity	High, noisy	Preserved (via representative raw log)

VII. RESULTS

We validated our real-time log–metric correlation framework on a distributed 8-GPU cluster running BERT pretraining alongside concurrent workloads. An Out-of-Memory (OOM) fault was induced on one GPU by progressively increasing the batch size beyond available memory. This scenario represents a common production failure mode in distributed training environments where memory exhaustion can severely impact training operations. The framework captured the full diagnostic sequence in real time: the DCGM_FI_DEV_FB_USED metric (gpu_memory) spiked sharply prior to the OOM event, while critical GPU error logs detailing memory allocation failures were generated concurrently. Critical OOM logs, shown in Fig. 6, from the affected GPU node (including CUDA stack traces and memory allocation failures) occurred concurrently with the gpu_memory anomaly. Dynamic Time Warping (DTW) aligned the metric trajectory with OOM log events, capturing strong temporal coupling despite phase shifts as shown in Fig 7. DTW accurately linked the error logs to the faulty GPU, establishing a precise causal chain from metric deviation to

fault origin without spurious correlations. The LLM-based RCA Copilot analysed the DTW-correlated logs in conjunction with GPU memory metrics (DCGM_FI_DEV_FB_USED) and statistical RCA signals to generate a comprehensive diagnostic summary.

```

"Sep 21 2025 08:24:49 keyword: error, pattern: unknown, ("app": "b13ca591-3b07-5da3-9217-716236434658", "cluster_name": "cto-lab", "gpu_uid": "GPU-c3526991-2da6-72b8-ecb8-0ad2e81c95b6", "layer_name": "nnccl", "log": "[rank2]: torch.cuda.OutOfMemoryError: CUDA out of memory. Tried to allocate 32.00 MiB. GPU 0 has a total capacity of 47.51 GiB of which 32.38 MiB is free. Process 2814707 has 40.12 GiB memory in use.")"

"Sep 21 2025 08:32:49 keyword: failed, pattern: unknown, ("app": "b13ca591-3b07-5da3-9217-716236434658", "cluster_name": "cto-lab", "gpu_uid": "GPU-c3526991-2da6-72b8-ecb8-0ad2e81c95b6", "layer_name": "nnccl", "log": "E0918 06:19:38.156000 3513965 torch/distributed/elastic/multiprocessing/api.py:870] failed (exitcode: 1) local_rank: 0 (pid: 3514067)*"
    
```

Fig. 6. Critical Logs Identified from Log-Metric Correlation Analysis

It identified the symptoms CUDA out-of-memory (OOM) errors and GPU framebuffer memory exhaustion pinpointed the implicated device, determined the root cause as GPU memory reaching maximum capacity during BERT pretraining due to excessive batch size under concurrent workloads, explained the underlying reason (batch size allocation exceeding available GPU memory), and recommended actionable remediation steps, including batch size optimization, memory-aware job scheduling with reservations, and enabling GPU memory pooling as shown in Fig. 8.

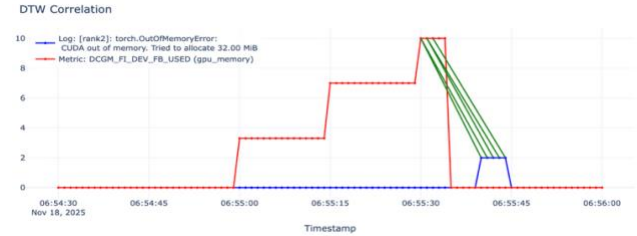


Fig. 7. DTW Correlation Between Metric and Log Timeseries

Symptoms

GPU Memory Exhaustion (OOM)

GPU framebuffer memory reached maximum capacity causing CUDA out-of-memory errors during training operations. Multiple concurrent jobs may have exceeded available GPU memory (DCGM_FI_DEV_FB_USED reached 100%), likely preventing tensor allocation.

Critical logs show memory allocation failures with process 2814707 having 40.12 GiB memory in use on a GPU with 47.51 GiB total capacity, leaving only 32.38 MiB free while attempting to allocate 32.00 MiB, suggesting resource exhaustion and job termination.

OOM Failures

Memory Capacity Reached

CUDA Allocation Errors

Actions

Optimize Memory Usage and Job Scheduling

- Review and adjust batch sizes across concurrent jobs
- Implement job scheduling policies with memory reservation
- Monitor per-job memory usage and set allocation limits
- Consider enabling GPU memory pooling and cache optimization strategies

DCGM_FI_DEV_FB_USED

GPU Memory Management

Job Scheduling

Resource Allocation

Fig. 8. Actionable Insights Using RCA Copilot enhancing by Log Analytics

VIII. CONCLUSION

This paper presents a real-time, topology-aware log–metric correlation framework that transforms large-scale logs into structured, causally relevant signals for automated RCA in hybrid multicloud systems. By combining template-based log reduction, multi-dimensional scoring, and temporally elastic correlation, the framework enables accurate cross-layer fault attribution, low-latency diagnostics, and reduced MTTR, providing a scalable and explainable foundation for LLM-assisted operations.

IX. REFERENCES

- [1] Z. Hu, S. Shen, T. Bonato, S. Jeaugey, C. Alexander, E. Spada, J. Dinan, J. Hammond, and T. Hoefer, "Demystifying NCCL: An in-depth analysis of GPU communication protocols and algorithms," arXiv preprint arXiv:2507.04786, 2025. [Online]. Available: <https://arxiv.org/abs/2507.04786>
- [2] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in Proc. IEEE 24th Int. Conf. Web Services (ICWS), Honolulu, HI, USA, Jun. 2017, pp. 33–40. doi: 10.1109/ICWS.2017.12
- [3] D. J. Berndt and J. Clifford, "Using dynamic time warping to find patterns in time series," in Proc. 3rd Int. Conf. Knowledge Discovery and Data Mining (KDD Workshop), Seattle, WA, USA, Jul. 1994, pp. 359–370
- [4] A. Ahmadi, S. Sharif, and Y. M. Banad, "MCP Bridge: A Lightweight, LLM-Agnostic RESTful Proxy for Model Context Protocol Servers," arXiv, Apr. 2025. [Online]. Available: <https://arxiv.org/abs/2504.08999>
- [5] K. Pearson, "Notes on regression and inheritance in the case of two parents," *Proc. R. Soc. Lond.*, vol. 58, pp. 240–242, 1895.
- [6] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *IEEE Transactions on Acoustics, Speech, and Signal Processing*.